

C-AIMMS Pipeline Teardown

How the IterRet + delta-Mem pipeline actually runs on LoCoMo, the state of the two-phase OSAM write path, an inventory of dataset-specific shortcuts, and an attribution of where the score is being lost.

Backbone	Qwen3-4B-Instruct-2507 + delta-mem adapter (TSW, token write granularity)
Retrieval	IterRet CTC graph traversal over vLLM, 5 iterations max
Benchmark	LoCoMo, 10 conversations, categories 1-4 (1540 questions)
Best measured	0.3016 overall / SH 0.3479 MH 0.2582 TEMP 0.2430 OD 0.2183
Working tree	Contains unvalidated changes; no full run has been produced from it

Contents

00. Where things stand	3
Why the run history looks incoherent	3
01. How one question is answered, end to end	4
02. What the runs actually say	7
Where the F1 mass is lost, newest run	7
03. The two-phase OSAM write path	8
Layer 2 - what was actually broken	8
The discovery that reframes it	8
Mapping the paper's strategies onto the config	8
How the existing adapter was trained	9
What was changed	9
Still open: there is no persistent memory	9
04. The A/B: does message_mean cost anything on TSW weights?	11
Design	11
Running it	11
What it reports	11
The decision this produces	11
Cost	12
05. Why single-hop and multi-hop stay low	13
06. Temporal: the one category actually regressing	15
The deeper problem: nobody is doing the arithmetic	15
Concrete improvements, in order of expected value	15
07. Hardcoding inventory	17
Group A - derived from inspecting the test set's answers	17
Group B - dataset-shaped but defensible	17
Group C - magic constants with no derivation	18
Group D - environment	18
08. Suggested order of work	19

00. Where things stand

The single most important fact

The code in the working tree has never produced a full result. `osam_workmem.py` and `eval_locomo_iterret_mock.py` were last edited 13 Aug 23:56; the newest full result file, `workmem_idf_plus_fusion.jsonl`, finished 13 Aug 19:53 - four hours earlier. Every number in this document describes the *previous* version of the answer prompt and evidence filter.

Three changes are unmeasured: the hard anti-abstention prohibition, the rescored Yes/No instruction, and `narrow_evidence_by_score_gap` (whose own comment states it has never been run end to end). The OSAM write-path fix and the A/B harness described in sections 03 and 04 are also unrun.

Why the run history looks incoherent

Two different denominators have been reported as "overall F1", and they differ by about +0.11. Runs 1-7 averaged over all 1986 questions *including category 5 (adversarial)*, which the system scored at 0.78-0.83 because a refusal is trivially easy to produce. From run 8 onward category 5 is excluded entirely - the eval skips it and writes no row - so the denominator is 1540 and the easy points are gone.

So "the score dropped from 0.389 to 0.294" is an artefact of the denominator, not a regression. On the like-for-like 1540-question basis the pipeline went **0.2756 to 0.3016**.

Correcting a premise

Single-hop and multi-hop are **not declining**. Across the last four comparable measurements single-hop went 0.3051, 0.2989, 0.3306, **0.3479**; multi-hop went 0.2192, 0.1858, 0.2412, **0.2582**. Both are rising. They are simply low.

The category actually regressing is **temporal: 0.3015 to 0.2430**, a loss of 0.0585 over three runs. Section 06 attributes it precisely.

01. How one question is answered, end to end

The stages below are a genuine sequence - each consumes the previous one's output. Where the code diverges from the architecture it documents, the divergence is called out inline.

A Launch and supervision

`run_full_pipeline.slurm -> guardian.sh`

Slurm starts a vLLM server on GPU 0 serving Qwen3-4B-Instruct-2507 at --max-model-len 8192, waits for /v1/models, then runs the eval on GPU 1. guardian.sh resubmits in a loop until the checkpoint file reaches 1540 lines, so a preempted job resumes rather than restarts.

Two copies of the model are live at once: vLLM (all retrieval reasoning) and a separate HuggingFace copy with the delta-mem adapter attached (answer generation only).

B Dialogue flattening

`eval_locomo_iterret_mock.py:144-154`

Every turn becomes a DialogueTurn whose text is "[session_ts] speaker: text" and whose time is the same session timestamp. All turns in a session share one timestamp; LoCoMo has no per-turn times.

Divergence - Data loss

Only dialog["text"] is read. The blip_caption field is dropped, though LoCoMo's own protocol renders it (locomo_protocol.py:247). 20.8% of turns carry a caption and 37.8% of gold-evidence references point at a captioned turn. Turns whose entire content is the image ("take a look at this") become contentless nodes with no retrievable cues.

C CTC graph construction

`iterret/memory_builder.py:134-171, cached to graph_cache/`

Three LLM-driven passes build the Cue-Tag-Content graph: one call per turn extracts a tag plus 2-6 cues and links each cue to the episodic node; a semantic pass over 4000-char chunks emits s* fact nodes (linked); a topic pass emits t* nodes. Roughly 600 vLLM calls per conversation, cached so a crash does not repeat them.

Divergence - Dead layer

Topic nodes are created with add_content() but never link(ed) to any cue or tag. Since traversal only reaches content through (cue, tag) edges, no topic node can ever be retrieved. They still enter the IDF document-frequency corpus, so they skew the term weights of a layer that can never be returned. Their reader, topic_to_episodes(), has no caller.

Divergence - Text duplication

memory_builder re-prefixes the speaker onto text that already contains it, and ContentNode.display_text() re-prefixes the timestamp. Every evidence string reads "[1:56 pm on 8 May, 2023] Caroline: [1:56 pm on 8 May, 2023] Caroline: I went to...". Lexical scoring is set-based so it is unaffected, but every embedding is computed over a string whose first half is boilerplate shared by every node in the conversation, which compresses cosine separation, and the OSAM token budget pays for it twice.

D Experience bank - instantiated empty

`eval_locomo_iterret_mock.py:173-184`

An ExperienceBank is built with a MiniLM-L6-v2 backend and deliberately left empty (run 3 showed a bank built from one conversation hurt the other nine). The same backend is attached to the graph as its embedder, which is what activates RRF fusion in rank_contents_by_relevance.

Divergence - Consequence

R2-Mem's Planning/Reflection experience retrieval - IterRet's headline novelty - contributes nothing to any current number. What runs is MRAgent-style CTC traversal plus a reflection loop. abstract_situation() is still called unconditionally before each empty bank.retrieve(): 2 wasted LLM round trips per iteration, up to 10 per question, roughly 15,000 per full run, producing a string that is embedded into nothing.

E IterRet retrieval loop

workmem/iterret_bridge.py -> iterret/nodes.py

Category 5 questions are skipped outright. For the rest, up to 5 rounds of: **retrieve_node** - match query to cues (a cue matches only if every one of its tokens appears in the query, capped at 40); situation abstraction (wasted); action-selection LLM call; traverse cue to tag to content masked against visited ids; cap tags at 15 by IDF-weighted overlap and new content at 25 by reciprocal-rank fusion of IDF-lexical and MiniLM-cosine order.

reflect_node - situation abstraction (wasted); routing LLM call returns kept_content_ids, resolved/new gaps and a refined query. If it answers "ALL", is unparseable, or returns ids that do not exist, the batch is silently replaced by the embedding top-6.

Stops on token budget (6000), two consecutive rounds with nothing new, no remaining gaps, 2 stuck reflects, or 5 iterations. 4 LLM calls per round, up to 20 per question.

Divergence - The recall safety net is dead code

semantic_fallback_contents() and semantic_match_cues() are fully implemented and documented as the recall safety net for when cue-gated traversal surfaces nothing - and have zero callers. When the strict-subset cue match returns nothing, the loop produces no evidence and the question scores a hard 0.

Divergence - Arbitrary tie-breaking at the cap

rank_tags_by_relevance sorts by (-score, tag). Most tags share no stopword-stripped token with the query and score exactly 0.0, so the 15-tag cap keeps whichever zero-scoring tags sort alphabetically first. Content ties break on id, where e1 < e10 < e100 < e11 - a lexicographic bias toward early-session turns.

F Evidence filtering and ordering

workmem/evidence_filter.py, osam_workmem.py:97

filter_evidence_by_relevance drops strings under 3 words and sorts everything by cosine similarity to the question, descending. Its threshold=0.30 parameter is accepted and never used - no filtering happens, only reordering. The 3-word floor never fires either, because every string starts with a roughly 6-word timestamp prefix.

In the working tree only, temporal-looking questions then pass through narrow_evidence_by_score_gap, which cuts at the first similarity gap exceeding that question's own mean+stdev.

Divergence - Chronology destroyed

Sorting by relevance means the model receives the conversation's events out of time order. For a benchmark where 80% of temporal gold answers are absolute dates and the answer usually depends on which session a statement came from, a shuffled timeline actively fights the task.

G OSAM Phase 1 - evidence write

osam_workmem.py -> runtime/session.py -> core/delta_impl.py

A fresh DeltaMemChatSession is created per question, reset, granularity set for Phase 1, and each evidence string written as its own message, then tokenized and ingested. Section 03 covers this stage in full - it is where the two-phase protocol lives and where it was silently failing.

H OSAM Phase 2 - answer generation

osam_workmem.py:145-272

Granularity flips to "token". Three instructions are always prepended (no first-person pronouns; the answer policy; the Yes/No scoping rule), a fourth is appended when the temporal regex fires, and the official LoCoMo QA prompt is formatted last. Decoding is greedy.

Divergence - Generation length

answer_with_osam imports OFFICIAL_MAX_NEW_TOKENS (50) and never uses it, and the eval calls it with no max_new_tokens, so generate_reply's default of 2048 applies. The two other eval scripts in the repo both pass 50. Score impact is small - only 6 predictions exceed 50 words - but one multi-hop answer ran to 1371 words, and the protocol's sampling parameters (temp 0.4 / top-p 0.9 / top-k 10) are likewise defined and unused.

I Scoring and checkpointing

deltamem/eval/locomo_protocol.py:124

Category 1 is scored with `multi_answer_f1` (both sides comma-split; for each gold part, the best-matching prediction part; averaged). Everything else uses `single_answer_f1` - Porter-stemmed, article-stripped token F1. Category 3 keeps only the text before the first semicolon. One JSON row is appended per question.

Divergence - No retrieval instrumentation

The row stores `n_evidence_retrieved` but not the evidence itself and not the gold evidence ids. It is therefore impossible to tell, from any result file that exists, whether a zero came from retrieval missing the fact or the reader failing to use it. That is the single biggest reason eight runs have not converged on a diagnosis.

02. What the runs actually say

Result file	Rows	Incl. cat5	Excl. cat5	Multi-hop	Temporal	Open-dom	Single-hop	Advers.
run1	1986	0.3548	0.2310	0.2068	0.1124	0.0980	0.2995	0.7825
run2	1986	0.3746	0.2441	0.2221	0.1990	0.0978	0.2855	0.8251
run3	1985	0.3733	0.2403	0.2005	0.1750	0.1038	0.2942	0.8337
run4	1986	0.3594	0.2238	0.1842	0.2348	0.1358	0.2430	0.8274
run5	1986	0.3701	0.2396	0.2000	0.1939	0.0706	0.2896	0.8206
run6	1986	0.3894	0.2756	0.2192	0.3015	0.0960	0.3051	0.7825
run7	1986	0.2042	0.2633	0.1858	0.2900	0.0896	0.2989	0.0000
run8 (workmem_iterret_full)	1540	-	0.2935	0.2412	0.2697	0.2014	0.3306	-
idf_plus_fusion (newest)	1540	-	0.3016	0.2582	0.2430	0.2183	0.3479	-

Run 7's adversarial score of 0.0000 is where the anti-abstention instruction was introduced: the model stopped refusing, which is the correct answer for every category-5 question. That is why category 5 was excluded from run 8 onward rather than fixed.

Where the F1 mass is lost, newest run

Failure mode	Single-hop	Multi-hop	Temporal	Open-domain
Zero, wrong content	26.6% / 40.8%	25.2% / 33.9%	22.4% / 29.6%	26.0% / 33.3%
Abstention ("not mentioned")	15.9% / 24.1%	10.6% / 14.0%	20.9% / 27.6%	17.7% / 22.7%
Partial credit under 0.5	11.7% / 13.0%	25.5% / 25.1%	22.7% / 19.6%	7.3% / 8.0%
Bare "Yes" / "No"	5.6% / 7.7%	7.4% / 6.7%	9.7% / 12.3%	38.5% / 33.5%
Verbose (precision loss)	4.4% / 5.7%	6.4% / 7.4%	0.9% / 1.1%	1.0% / 1.0%
Empty / skipped	0.7% / 1.1%	3.2% / 4.3%	0.9% / 1.2%	1.0% / 1.3%

Read each cell as *share of questions / share of that category's total lost F1*. Bare Yes/No consuming a third of open-domain's loss is the clearest single anomaly in the table.

03. The two-phase OSAM write path

This is the section that changed most. "Is the two-phase working" turns out to be three independent questions, and they fail separately. Debugging effort had been concentrated on layer 1, which was never the problem.

Layer	Question	Status
1. Routing	Does the code <i>select</i> message_mean for Phase 1 and token for Phase 2?	Always fine. The caller was correct.
2. Plumbing	Does the selected granularity actually <i>execute</i> in the engine?	Was broken. Now fixed and guarded.
3. Weights	Are the adapter's weights <i>trained</i> for the granularity selected?	Unresolved. Section 04 measures it.

Layer 2 - what was actually broken

set_delta_mem_write_granularity(model, "message_mean") is only half the contract. The engine also needs a per-token write_message_ids tensor telling it which tokens belong to which segment. Phase 1 wrote evidence as role="system" messages, and session.py:169 stamps message_id = -1 on every system message - upstream treats "system" as out-of-band scaffolding, not memory content. The chain:

```
all evidence -> message_id = -1
delta_impl.py:1410 active_mask = message_ids.ge(0) -> empty
delta_impl.py:1413 return None
delta_impl.py:2184 else: token-granularity scan <- SILENT FALLBACK
```

Phase 1 collapsed into Phase 2. And it was invisible: populate_osam_from_evidence returns state_stats() to "verify S changed", and S *did* change - via the fallback. That check cannot distinguish the two paths, which is why the bug survived many sessions.

Corroboration from upstream

delta-Mem's own reference implementation agrees. build_teacher_forced_snapshot receives its history from build_locomo_history_messages, which emits **one** system message (the instruction) and every unit of memory content as **role="user"**. The osam_workmem docstring claims to reuse that ingest pattern; it had inverted it.

The discovery that reframes it

The adapter was trained with token writes

The model card in the repo (models/delta-mem-adapter/README.md) states: **Variant TSW, Token State Write. Write granularity: token.**

So the layer-2 bug was forcing the pipeline into token writes - which is exactly what those weights expect. **All eight runs were train/inference consistent, by accident.** Fixing layer 2 exposes layer 3: before the fix, the wrong plumbing was compensating for the wrong weights.

Why this is a weights problem and not a settings problem: memory_k_proj and memory_v_proj learned to map *individual token hidden states* into memory writes. message_mean averages a whole evidence item's hidden states into one vector first. An average has different magnitude and direction than any real token - an input type those weights never saw. Routing logic decides which door the data goes through; both doors lead to the same weights.

Mapping the paper's strategies onto the config

Paper	Full name	Config	Meaning
TSW	Token-State Write	memory_write_granularity="token"	Updates S at every token position
SSW	Sequence-State Write	memory_write_granularity="message_mean"	Averages hidden states within each message; each message updates S once
MSW	Multi-State Write	num_state_heads=4 (granularity stays "token")	Parallel sub-states, each accumulating different information

The two-phase split is not in the paper

arXiv 2605.12357 applies one write strategy uniformly through ingestion and generation. There is no segment-write-then-token-generate protocol in it. Two-phase OSAM is C-AIMMS's own design. That is legitimate as a contribution, but it means there is no published SSW-write/token-generate result to lean on, and the paper's ablation (TSW 51.66, SSW 51.44, MSW 50.74) measures each strategy *trained for itself* - not TSW weights driven at SSW granularity, which is none of those three conditions.

How the existing adapter was trained

The backbone is never trained. `freeze_non_delta_mem_params` sets `requires_grad=False` on every parameter, then re-enables only the non-base parameters inside each `DeltaMemAttention` wrapper: `memory_q/k/v_proj` (the translator itself), `delta_q_proj` and `delta_o_proj` (enabled by `--delta-heads q,o`), plus a handful of gates and scales. The saved artefact is `config.json` plus `delta_mem_adapter.pt` containing exactly those tensors - a few MB. It is not a PEFT LoRA and must not be merged.

Data	<code>agent_memory_qasper_ctx8192_episode_safe_seed42.jsonl</code> - Qasper, academic-paper QA, episode-formatted, roughly 6k examples. Not present in this repo and no script builds it.
Mode	<code>--training-mode episode</code> , <code>--assistant-loss-mode final_assistant_only</code> , <code>--episode-recent-messages 1</code>
Split	<code>--max-length 512</code> (backbone input) vs <code>--max-write-length 8192</code> (written into S)
Memory	<code>rank 8</code> , <code>alpha 16</code> , <code>--delta-heads q,o</code> , <code>--num-state-heads 1</code> , <code>--memory-write-granularity token</code> , <code>--memory-write-source learned_hidden</code>
Aux losses	<code>contrast 1.0</code> , <code>KL 0.02</code> , <code>margin 0.01</code> , <code>causal 1.0</code> , <code>anchor 1.0 / 0.005</code> , <code>recover 0.25</code> , <code>need-floor 0.15</code> , <code>state-only dropout 0.2</code> , <code>write-sparsity</code>
Optim	<code>lr 2e-4 cosine</code> , <code>warmup 0.10</code> , <code>wd 0</code> , <code>adamw_torch_fused</code> , 1 epoch, <code>bs 1 x grad-accum 4 x 8 GPUs</code> (effective 32), <code>bf16</code> , <code>seed 42</code>

The 512 / 8192 split is the paper's mechanism made concrete: up to 8192 tokens of context go into S, only one recent message plus the question stays in the backbone's input, and loss lands on the final assistant turn only. The adapter is trained to answer *from* S. Note the domain: Qasper is single-document academic QA, not conversation, so the adapter is already out of domain for LoCoMo before any of these changes.

What was changed

- **role="system" changed to role="user"** in `populate_osam_from_evidence`. Each evidence item now receives a real message id, so `message_mean` pools one write per retrieved unit. Required for any segment granularity to work; inert under token.
- **`_assert_segment_write_fired` added**. Runs after every Phase 1 ingest and raises if the granularity did not reach the modules, if `_runtime_write_span_ids` returned `None` (a second silent-fallback path, triggered when the chat template is not prefix-stable), if the write mask is empty, or if the number of write segments does not equal the number of evidence items. That last check is the real guarantee - it proves one write per evidence unit. Skipped under token, where there is no mask to verify.
- **OSAM_PHASE1_GRANULARITY environment flag added** (default `message_mean`), so token and `message_mean` can be compared without editing files.

Net effect

The two-phase protocol now genuinely executes and can no longer degrade silently. Whether it *should* execute at `message_mean`, given a TSW-trained adapter, is the open question that section 04 exists to answer.

Still open: there is no persistent memory

Independent of granularity, a fresh session is built per question and S is written only from that question's retrieved evidence. `delta-Mem`'s own LoCoMo eval does the opposite: `_build_sample_cache` builds one `full_snapshot` per conversation via `build_teacher_forced_snapshot` over the entire history, then for every question restores that `delta_state` and also replays the full history in context (the condition is named `full_history_replay`). S is therefore persistent across all of a conversation's questions upstream, and rebuilt from scratch here.

DeltaMemChatSession already exposes `snapshot()` and `load_snapshot()` for exactly this. The restructure is to hoist OSAM population to the sample level, snapshot it, restore per question, and layer the retrieved evidence on top with `reset=False`. Cost: one forward pass over roughly 25-35k tokens per conversation, ten times. This applies whichever way the A/B lands.

04. The A/B: does message_mean cost anything on TSW weights?

Harness: `deltamem/workmem/ab_write_granularity.py`. Purpose: turn the layer-3 question into a number, so the decision to ship two-phase (or to train an SSW adapter first) rests on measurement rather than argument.

Design

Retrieval runs **once per question**, and both arms generate from the identical evidence list in the identical order. Phase 2 is "token" in both arms - `answer_with_osam` sets that itself. Decoding is greedy in both. So the only variable anywhere in the pipeline is the Phase 1 write granularity.

Held constant	Varied
Retrieved evidence (identical list, identical order)	Phase 1 write granularity only:
Evidence filter and temporal narrowing	token (TSW - matches the adapter's training)
Answer prompt and instruction stack	vs message_mean (SSW - what two-phase calls for)
Phase 2 granularity ("token")	
Greedy decoding, same model, same adapter	

This buys two things. The comparison is **paired** - every question contributes a matched pair, so the test measures a within-question difference rather than comparing two noisy means, which is far more sensitive at n around 150. And it costs roughly **half** of running the eval twice, because retrieval (about 20 vLLM calls per question) dominates and is now shared.

Running it

The vLLM server must be up, exactly as for the main eval.

```
CUDA_VISIBLE_DEVICES=1 \
AB_OUTPUT_FILE=/home/kbasu/arnavbhatt/workmem_test/outputs/ab_write_granularity.jsonl \
python3 -u -m deltamem.workmem.ab_write_granularity
```

Point `AB_OUTPUT_FILE` into the existing `outputs/` directory as above: the script derives its graph cache from that path's parent, so it reuses `outputs/graph_cache/sample_0.json` and skips the roughly 600-call graph build. It is resumable - rerun after a preemption and it continues where it stopped.

Variable	Default	Meaning
<code>AB_MAX_SAMPLES</code>	1	How many conversations to cover
<code>AB_OUTPUT_FILE</code>	<code>outputs/ab_write_granularity.jsonl</code>	Resumable checkpoint; also sets the graph-cache directory
<code>AB_ARMS</code>	<code>token,message_mean</code>	Granularities to compare; the first is the baseline
<code>AB_MAX_NEW_TOKENS</code>	unset	Unset reproduces production generation length

What it reports

```
PAIRED A/B -- Phase 1 write granularity (n = 148 questions)
      token F1 = 0.xxxx
      message_mean F1 = 0.xxxx

mean paired diff (message_mean - token) = +0.0xxx
95% bootstrap CI                        = [-0.0xxx, +0.0xxx]
message_mean better on N, worse on M, tied on K
identical predictions: N/148
per category: ...
VERDICT: ...
```

The confidence interval is a paired non-parametric bootstrap (10,000 resamples, seeded, no scipy dependency) over the per-question score differences. It was validated against synthetic data before shipping: the bootstrap mean equals the sample mean exactly, the interval width tracks 4x the standard error, and it covers zero under the null across independent seeds.

The decision this produces

Outcome	Reading	Action
CI brackets zero	TSW weights tolerate segment writes; the mismatch costs nothing measurable	Ship two-phase now. Note the train/inference caveat in the paper. Question closed.
CI entirely below zero	The weights do not transfer to pooled writes	Two-phase needs an SSW adapter: request declare-lab's checkpoint (they report SSW 51.44, but only TSW is public), or rebuild the Qasper episode dataset and train. Alternatively run single-phase token and drop the two-phase claim.
CI entirely above zero	message_mean helps despite the mismatch	Ship two-phase and investigate why - this would be a genuinely interesting result.
All predictions identical	Tripwire: both arms took the same code path	Layer 2 is still broken; the role="user" fix did not take effect. Check populate_osam_from_evidence before reading any scores.

Built-in safeguards

Each row records **state_stats** per arm, so it can be confirmed numerically that the two arms produced genuinely different memory states rather than trusting the scores alone. The report prints an explicit warning if every prediction matches across arms. Questions that retrieve no evidence are skipped rather than counted, since both arms would score 0 identically and would dilute the paired difference toward zero.

Cost

Dominated by two generations per question rather than by retrieval. With the graph cached, expect roughly the wall clock of one smoke-test pass plus a margin - far below the half day the training path would cost, and it may remove the need for training entirely.

05. Why single-hop and multi-hop stay low

CRITICAL

SH-1 Abstention is the largest addressable loss, and the fix is untested

osam_workmem.py:204-221 (working tree); measured on workmem_idf_plus_fusion.jsonl

134 single-hop and 30 multi-hop answers are refusals, together 24.1% and 14.0% of those categories' lost F1. They score a mean of 0.013 and 0.023 - effectively nothing. Real cases: "How long have Mel and her husband been married?" answered "Cannot determine" with 16 evidence items retrieved; "What did Caroline research?" answered "Nothing mentioned about Caroline researching" with 37 items retrieved.

The hardened prohibition in the working tree targets this and is the right instinct, but it has never been run, and the softer version it replaces moved abstention from 15.3% (run 6) to 16.1% - i.e. not at all. Prompt pressure alone has a poor track record here on a 4B backbone.

Suggested: a second pass rather than a stronger instruction - detect a refusal in the decoded string and re-ask once with "the answer is present in the evidence; name it". Converts a 0.01 into a real draw at the cost of one extra generation on about 16% of questions.

CRITICAL

SH-2 The model paraphrases; the metric only rewards quotation

single-hop precision 0.358 / recall 0.375 against gold mean length 4.2 tokens

The "zero, wrong content" bucket - 224 single-hop questions, 40.8% of the category's entire loss - is not mostly retrieval failure. Most are semantically correct answers phrased in the model's own words:

gold "she thinks Caroline is doing something amazing"	pred "Supportive"
gold "love, faith, and strength"	pred "Freedom and being real"
gold "art and self-expression"	pred "Relaxation and creativity"
gold "a cup with a dog face on it"	pred "No answer available"

The official prompt already says "Answer with exact words from the conversations whenever possible", but it is the *last* line of a four-instruction stack with three project-authored instructions above it competing for adherence. Gold-answer tokens are present in the gold evidence turns 82.5% of the time for single-hop, so the ceiling is high and the reader is what loses it.

Suggested: promote span-copying to the first instruction and make it imperative - "Copy the exact words from the evidence. Do not paraphrase, summarise or generalise." Metric-shaped but not test-set-derived, and the highest-expected-value single change available.

CRITICAL

MH-1 Multi-hop is scored on lists, and the prompt asks for one thing

locom_protocol.py:113 multi_answer_f1; OFFICIAL_QA_PROMPT

Category 1 is the only category scored with multi_answer_f1, which comma-splits both sides and averages the per-gold-part best match. 169 of 282 multi-hop golds have two or more parts. On those:

Prediction lists at least as many parts as the gold	n = 49	mean F1 0.356
Prediction lists fewer	n = 120 (71%)	mean F1 0.197

Answering "Painting" when the gold is "Running, pottery" caps the score at 0.5 by construction. The prompt causes this directly: "write a short answer in a few words. Do not write complete and lengthy sentences." Closing the gap on those 120 questions is worth roughly **+0.07 on multi-hop** on its own.

Suggested: for multi-part-shaped questions only (plural head nouns, "What things...", "Which...", detected from question text and never from the category label), add: "If more than one thing satisfies the question, list all of them separated by commas."

HIGH

SH-3 Bare "No" appeared between run 7 and run 8 and was never removed

osam_workmem.py:226-230

Bare yes/no answers went from 2 in run 7 to 128 in run 8 and 136 in the newest run, because Yes/No became the first worked examples in the judgment instruction. On open-domain this is now 38.5% of questions and 33.5% of all lost F1; on single-hop, 47 questions. Real case: "What are the new shoes that Melanie got used for?" answered "No" (gold: "Running").

The working tree's rescoped wording addresses this and looks correct, but sits in the unmeasured bucket, so this remains an open regression until a run lands. Cheapest confirmed-cause item on the list.

HIGH

R-1 Retrieval is running with its recall floor amputated

iterret/ctc_graph.py:365-404; iterret/nodes.py:84-151

A cue matches only if every token of the cue appears in the question - a deliberate fix for context blowup, but a hard gate with no fallback wired up. semantic_match_cues and semantic_fallback_contents exist, are documented as the safety net, and are never called. When the gate returns nothing the question gets no evidence and a hard zero.

Suggested: union semantic_fallback_contents(query, top_k) into the traversal when cue-gated traversal yields fewer than k new items. Already written; only the call is missing, and the embedder is already attached.

HIGH

R-2 Image captions are 21% of the corpus and are never ingested

eval_locomo_iterret_mock.py:151 vs locomo_protocol.py:247

1226 of 5882 turns carry a blip_caption; 1064 of 2815 gold-evidence references (37.8%) point at one. Including captions raises gold-answer token coverage inside gold evidence by +0.029 on single-hop and +0.026 on open-domain - a ceiling raise before any retrieval improvement. The larger effect is that captioned turns currently generate cues from text like "By the way, take a look at this", so they are effectively unindexed.

Suggested: append the caption in stage B exactly as the official protocol does. Invalidates the graph cache, so budget a full rebuild.

CRITICAL

D-1 Retrieval failures cannot currently be distinguished from reader failures

eval_locomo_iterret_mock.py:280-284

Every diagnosis in this section had to be inferred from prediction text, because result rows contain no evidence and no gold-evidence ids. Eight runs of prompt tuning have been flying blind on the question that matters most: of the 224 single-hop zeros, how many had the answer in the retrieved evidence?

Suggested: add three fields per row - evidence (the list of strings), gold_evidence (from question["evidence"]), and evidence_ids. Cost is disk. Payoff is that the next run answers the recall-vs-reader question definitively.

06. Temporal: the one category actually regressing

Temporal fell 0.3015 to 0.2430 across runs 6, 7, 8 and the fusion arm. The mechanism is unambiguous once predictions are split by whether they contain a date at all.

Run	Temporal F1	Predictions containing a date	F1 given a date	Bare Yes/No	Mean words
run6	0.3015	73.2%	0.3995	0	14.7
run7	0.2900	70.7%	0.3974	0	15.4
run8	0.2697	37.4%	0.5312	27	3.1
fusion	0.2430	34.6%	0.4623	31	3.5

The terseness push worked exactly as designed - when the model emits a date it is now far more accurate (0.40 rising to 0.46-0.53). It simply stopped emitting dates, on half the questions that need one.

Root cause

Date-emission coverage collapsed from 73% to 35% and swamped the precision gain. Of the 258 temporal questions whose gold answer is an absolute date, 41.9% now produce a prediction containing neither a date nor even a relative expression - those score a mean F1 of 0.040. Terseness pressure plus the Yes/No exemplars turned "I am not sure of the date" into "No" instead of into a guessed date.

The deeper problem: nobody is doing the arithmetic

LoCoMo attaches one timestamp per *session*, and the answer is usually anchored to a relative phrase inside a turn. Of the 130 temporal questions where both the gold and its evidence session date parse as full dates:

53%	gold equals the session date exactly (69 questions)
32%	gold is session date minus one day - "yesterday" (41 questions)
9%	minus two or more days (12 questions)
0.183 -> 0.591	gold-token coverage in evidence text alone, versus once the session date is included

The canonical case is the very first question in the dataset. "When did Caroline go to the LGBTQ support group?", gold **7 May 2023**. The evidence turn is D1:3, "I went to a LGBTQ support group **yesterday**", in a session stamped "1:56 pm on 8 May, 2023". The model answered **"8 May, 2023"** - it copied the stamp instead of subtracting a day. Because token-overlap F1 gives partial credit for matching "may" and "2023", that wrong answer scores **0.667**, which is why this class of error has stayed invisible across eight runs.

The current instruction asks a 4B model to do calendar arithmetic in-context, on a shuffled timeline, over strings where the timestamp appears twice. That is close to the worst possible framing of the task.

Concrete improvements, in order of expected value

1. Resolve relative dates deterministically at ingestion, not in the model

In stage B, scan each turn for yesterday / last week / last month / two days ago / this morning and append the computed absolute date to the turn text: "... yesterday [= 7 May 2023] ...". Python does the arithmetic; the model only has to copy. Directly targets the 32%+9% offset cases and needs no prompt change.

2. Restore date-emission coverage without losing terseness

For gated questions make the format mandatory rather than the reasoning: "Answer with a date in the form D Month YYYY. If the exact day is not stated, give the closest date supported by the evidence. Never answer Yes or No." Coverage is worth more than precision here - 73% x 0.40 beats 35% x 0.53.

3. Order evidence chronologically for gated questions

Undo filter_evidence_by_relevance's relevance sort for temporal questions and sort by session timestamp instead. Relevance order stays the right default for the other categories; for "when", the timeline is the signal.

4. Emit the timestamp once, and label it

Fix the stage-C duplication so each item reads "[8 May 2023] Caroline: ..." once. Consider a leading "CONVERSATION DATE:" label so the stamp is unambiguously the recording time, not the event time - the copied-session-date error suggests the model conflates them.

5. Reconsider the score gate before shipping it

narrow_evidence_by_score_gap is applied only to temporal questions on the grounds that temporal shows dilution ($r = -0.07$). But the newest run already shows temporal's mean evidence count falling (11.1 to 8.9) alongside its score. Narrowing further pushes in the same direction as the regression, and its author's note says it has never been run. Gate it behind a flag and measure it as an arm, not as a default.

07. Hardcoding inventory

Grouped by how much damage each kind does to a claim of generality. Group A is the one that would not survive review.

Group A - derived from inspecting the test set's answers

CRITICAL

A-1 Gold answer strings are written into the answer prompt

osam_workmem.py:226-230

The Yes/No instruction names three examples: 'Pomodoro technique', 'psychology', 'the beach'. All three are **verbatim LoCoMo gold answers**. "pomodoro technique" is the exact gold for one question and appears in a second; "psychology" leads the gold for another; "beach" appears in nine. Under a token-overlap metric, seeding the prompt with gold tokens can earn score directly.

Magnitude is small - roughly a dozen questions - but the practice is not defensible in a paper and is invisible unless someone greps the golds.

Fix: replace with exemplars from a domain the benchmark does not cover, or drop the examples and keep the abstract rule.

CRITICAL

A-2 The abstention policy is justified by reading the answer key

osam_workmem.py:174-196

The comment states the reasoning plainly: abstaining is never correct here, because 0 of 1540 gold answers is a refusal, "checked directly against locomo10.json". The none/nothing prohibition is justified the same way. Both claims were verified and are true - that is exactly the problem: the policy is a decision made *because* of what the labels contain.

Fix: rejustify from the metric alone - under token-F1 with no abstention credit, guessing weakly dominates refusing for *any* question set. That argument is true a priori and needs no look at the labels.

HIGH

A-3 Category 5 exclusion: the mechanism is fine, the reporting is not

eval_locom_iterret_mock.py:211-214, 298-307; guardian.sh:20

Revised from an earlier, harsher assessment. delta-Mem's own reference eval sets DEFAULT_LOCOMO_CATEGORIES = (1, 2, 3, 4) with the help text "Defaults to 1 2 3 4, excluding adversarial", so excluding category 5 follows the upstream default and is not itself illegitimate.

What remains a real problem is the reporting: 446 of 1986 questions (22.5%) are dropped while the result is still labelled "OVERALL F1" with no qualifier, and the denominator changed mid-project, which is where most of the confusion in the run history comes from.

Fix: label the number "categories 1-4" everywhere, or route category 5 through answer_with_osam(..., allow_abstention=True) - the parameter already exists for exactly this - and report all 1986.

HIGH

A-4 Every ranking constant is tuned on the evaluation corpus

ctc_graph.py:24-48; osam_workmem.py:55-87

The stopword list is documented as "verified against this project's actual corpus (locom10.json)", with "may" deliberately excluded because 29 LoCoMo questions reference a May date. The narrowing gate's per-category thresholds come from correlations measured on the 1540-question run. These are honestly documented, which is to their credit, but they are still evaluation-set fitting with no held-out split - split_bootstrap_eval exists in locomo_data.py and the main eval never calls it.

Fix: fit on the 10% bootstrap split the code already knows how to produce; report on the remainder.

Group B - dataset-shaped but defensible

Item	Where	Assessment
_TEMPORAL_QUERY_RE	osam_workmem.py:29	Reads the question text, not the label - legitimate at inference time. Verified: fires on 86.0% of category-2 questions and on only 33 of the other 1219, matching the comment. English wh-shaped; will under-fire elsewhere. Honestly documented.
"No first-person pronouns"	osam_workmem.py:224	Exists because LoCoMo golds are third-person narration. Metric-shaped, not label-derived.
Episode exemplars "Pet Adoption", "Job Change"	memory_builder.py:18	Mildly suspicious - sample 0 of LoCoMo is literally about adoption. Not gold answers, but worth swapping for neutral examples.

Item	Where	Assessment
extract_session_nums / session_N_date_time	eval_locomo_iterret_mock.py:54	Fine. This is the dataset loader; it is supposed to know the schema.
Category 3 truncation at first semicolon	locomo_protocol.py:133	Part of the official protocol, not a project addition.

Group C - magic constants with no derivation

Constant	Value	Where	Note
MAX_ACTIVE_CUES	40	nodes.py:13	Never ablated.
MAX_ACTIVE_TAGS	15	nodes.py:15	Cap point where the alphabetical tie-break bites.
MAX_NEW_CONTENT_PER_ROUND	25	nodes.py:16	Never ablated.
FAIL_OPEN_FALLBACK_TOP_K	6	nodes.py:22	Governs how much evidence survives an unparseable routing reply - a common path.
_TOKEN_BUDGET	6000	iterret_bridge.py:29	Tied to --max-model-len 8192; the 1.35 words-to-tokens factor is a guess.
rff_k	60	ctc_graph.py:297	Conventional; fine.
max_matches / min_sim	25 / 0.2	ctc_graph.py:310, 366	min_sim sits on a function with no callers.
DEFAULT_MAX_CHARS_PER_CHARACTER	4000	memory_builder.py:44	Reasonable and documented.
threshold	0.30	evidence_filter.py:19	Accepted and never used. The eval passes it at call site 225, implying filtering that does not happen.
_MIN_WORDS	3	evidence_filter.py:5	Unreachable - the timestamp prefix alone exceeds it.
ITERRET_MAX_ITERATIONS	5	eval:26 vs eval:218	Constant defined, then the literal 5 passed at the call site. Changing the constant does nothing.

Group D - environment

Absolute cluster paths (/home/kbasu/arnavbhatt/workmem_test/...) are baked into all four eval scripts, all five slurm files, all three guardian scripts and score_calculator.py - nothing runs on another machine without editing. iterret/llm_client.py does this properly with ITERRET_LLM_BASE_URL and ITERRET_LLM_MODEL env overrides; the workmem side does not, so the eval's hardcoded VLLM_BASE_URL and VLLM_MODEL_NAME silently shadow that mechanism.

08. Suggested order of work

Sequenced so each step's result is interpretable: instrumentation first, then the questions that are cheap to settle, then the structural fixes that invalidate the graph cache, and training only if forced.

1. Log the evidence

Three extra fields per row (evidence, gold_evidence, evidence_ids). Until this lands, every conclusion about single-hop and multi-hop is inference from prediction text. Cheapest item on the list, largest effect on the ability to decide anything.

2. Run the A/B on Phase 1 granularity

Section 04. One sample, paired, free. Settles whether two-phase can ship on the existing TSW adapter or whether an SSW adapter is required - the question that currently blocks the architecture story.

3. Run the working tree once, as-is

Establish what the three unmeasured prompt changes actually did before layering more on. Use WORKMEM_OUTPUT_FILE so it does not collide with the existing checkpoint. Expect the Yes/No fix to help open-domain most.

4. Restructure to persistent, conversation-level memory

Hoist OSAM population to the sample level, snapshot, restore per question, layer retrieved evidence on top with reset=False. This is the mechanism that is currently missing entirely, and it is independent of the granularity question.

5. Fix the two temporal mechanics

Deterministic relative-date resolution at ingestion, plus a mandatory date format for gated questions. Targets the 41.9% of temporal questions emitting no date at all and the 32% needing a one-day subtraction.

6. Promote span-copying and add list-completeness

Single-hop's paraphrase loss and multi-hop's under-listing are the two largest reader-side buckets, worth roughly +0.07 on multi-hop alone.

7. Ingest blip_caption and de-duplicate the evidence strings

Both require a full graph-cache rebuild, so do them together - one 6000-call pass, not two.

8. Wire in the semantic fallback

Union semantic_fallback_contents into the traversal when it returns fewer than k new items. The code is written; only the call is missing.

9. Fix the group-A prompt leakage and settle the reporting denominator

Replace the gold-answer exemplars, rejustify the abstention policy from the metric, and label the headline number "categories 1-4" or restore category 5 via allow_abstention=True.

10. Train an SSW adapter - only if the A/B forces it

Requires rebuilding the Qasper episode dataset (absent from the repo, no build script), which is more work than the roughly half-day of training on 2 GPUs. If it comes to this, consider building the episode dataset from conversational data instead of Qasper, since LoCoMo is a conversation benchmark and the released adapter is already out of domain.

Scope note

Everything in this document was verified by reading the code and recomputing the metrics from the result files in outputs/. Statements about the delta-mem paper come from arXiv 2605.12357 and the repository's own README, model card and reference eval. No claim here rests on a run that has not happened: where a change is unmeasured, it is labelled as such.